Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Disciplina: Testes de Software
Trabalho: Análise de Eficácia de Testes com Teste de Mutação
Aluno: João Pedro Aguiar do Prado
Matrícula: 756491
Data: 10/05/2026

1. Análise Inicial

A avaliação inicial da suíte de testes da biblioteca `operacoes.js` revelou um cenário clássico de falsa sensação de segurança. Ao executar o framework Jest com a flag de cobertura, obtivemos uma cobertura de código (Code Coverage) de 78.11%, com 100% dos 50 testes passando com sucesso.

No entanto, ao submeter o projeto à análise do StrykerJS, o Mutation Score inicial foi de apenas 73.71%**. Dos 213 mutantes gerados, 44 sobreviveram.

Essa discrepância evidencia que a cobertura de código mede apenas a execução das linhas, mas não a qualidade ou o rigor das asserções (`expects`). Os testes originais percorriam o código, mas eram frágeis: não validavam estados de fronteira, aceitavam mensagens de erro genéricas e utilizavam massas de dados viciadas, permitindo que falhas lógicas fossem introduzidas sem quebrar a pipeline de integração.

2. Análise de Mutantes Críticos

Durante as execuções, o console do Stryker evidenciou padrões críticos de mutantes sobreviventes que mascaravam comportamentos indesejados no sistema. Em vez de depender apenas de interface gráfica, a análise foi fundamentada nos logs brutos gerados pela ferramenta. Destacam-se três casos representativos:

2.1. O Mutante Silencioso (Validação de Exceções)

A Mutação (Log): `- if (b === 0) throw new Error('Divisão por zero não é permitida.');` / `+ if (b === 0) throw new Error("");`

A Falha: O teste original verificava apenas se *alguma* exceção era lançada genérica e abstratamente (`expect().toThrow()`). O mutante sobrevivia pois a alteração (ou deleção) da mensagem da regra de negócio não quebrava o teste, o que poderia gerar falhas de integração com interfaces de usuário.

2.2. A Falsa Fronteira (Operadores Relacionais)

A Mutação (Log): `- function isMaiorQue(a, b) { return a > b; }` / `+ function isMaiorQue(a, b) { return a >= b; }`
A Falha: A suíte original não implementou Análise de Valores Limite (BVA). Como os testes usavam apenas valores com diferenças absolutas, o mutante que alterou a verificação estrita para inclusiva (`>=`) passou despercebido, pois faltava um cenário testando o limite de igualdade.

2.3. A Falsa Ordenação (Mascaramento de Lógica)

A Mutação (Log): `- const sorted = [...numeros].sort((a, b) => a - b);` / `+ const sorted = [...numeros].sort(() => undefined);`
A Falha na Função Mediana: O teste original fornecia à função um array previamente ordenado. Ao mutar (destruir) o callback do método `.sort()`, o retorno da mediana continuou correto, indicando que a suíte estava viciada e não forçava a execução real da lógica de ordenação antes do cálculo.

3. Metodologia de Solução e Ciclo de Iterações

Para mitigar essas vulnerabilidades, a abordagem de refatoração abandonou edições ad-hoc em favor de um pipeline iterativo e documentado. O processo foi segmentado em cinco rodadas principais, cada uma focada em um grupo lógico de mutantes, cujos dados e testes implementados encontram-se devidamente registrados nos arquivos de histórico anexos ao projeto (`historico_mutantes_iteracao_X.md`):

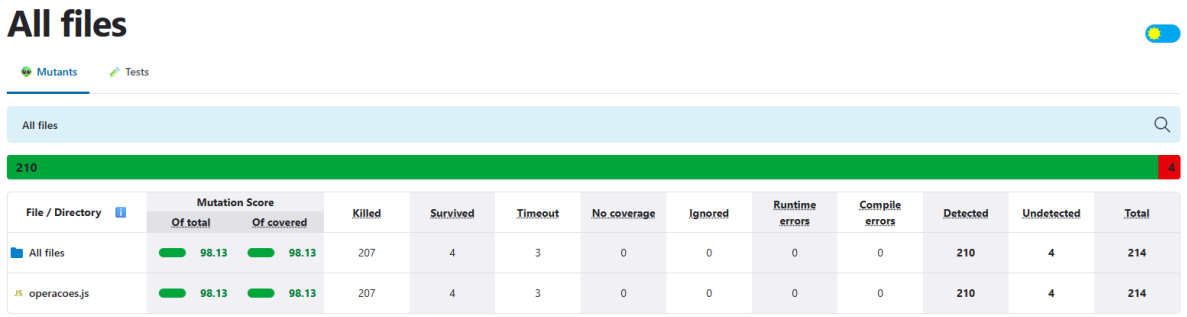
- **Iteração 1 (Base e Exceções):** Foco na eliminação de "falsos positivos" em funções com tratamentos de erro (como `dividir` e raiz quadrada). Foram implementadas asserções estritas que exigem a validação da mensagem de erro exata estipulada pela regra de negócio.
- **Iteração 2 (Operadores e Fronteiras):** Refatoração das funções matemáticas que utilizam operadores relacionais. Introdução de testes focados em casos de fronteira (valores idênticos) para funções como `isMaiorQue` e `isMenorQue`, garantindo a eliminação de mutantes lógicos e condicionais.

- **Iteração 3 (Laços e Condições de Parada):** Fortalecimento dos testes que cobrem iterações, como verificação de números primos e cálculos de fatorial. A suíte foi ajustada para capturar mutações que alteravam inicializadores de laços (`let i = 2`) ou suas condições de parada.
- **Iteração 4 (Validação de Regras e Arrays):** Foco em funções estatísticas e manipuladores de array. Aprimoramento da massa de testes para capturar comportamentos erráticos em arrays vazios ou validações matemáticas complexas (como clamp de valores entre mínimos e máximos).
- **Iteração 5 (Desviciamento de Dados):** Ajustes finais em funções de ordenação e cálculos dependentes de posição (como a `mediana`). Injeção de massas de dados dinâmicas e desordenadas para forçar o sistema a executar todas as etapas internas do processamento, eliminando os últimos mutantes de mascaramento lógico.

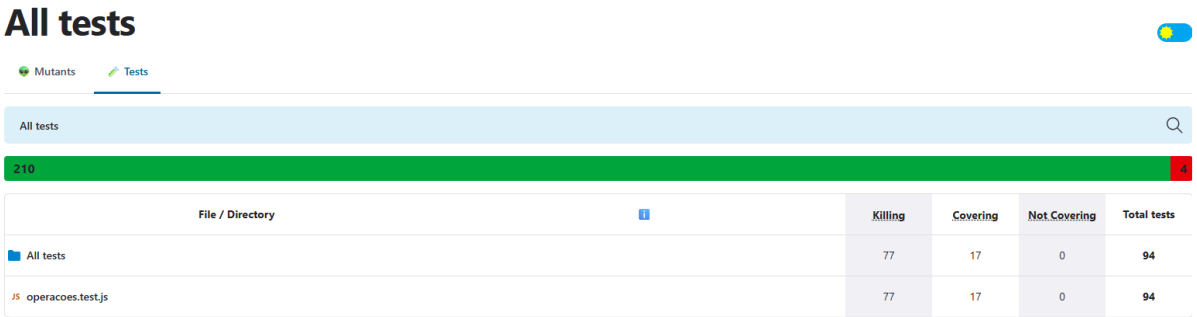
4. Resultados Finais

Após a refatoração iterativa da suíte de testes baseada no relatório de mutações, alcançamos os seguintes resultados:

Testes Passando: 100%
Mutation Score Final: 98.13%
Mutantes Eliminados: 209/214



File / Directory	Mutation Score Of total	Mutation Score Of covered	Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
All files	98.13	98.13	207	4	3	0	0	0	0	210	4	214
operacoes.js	98.13	98.13	207	4	3	0	0	0	0	210	4	214



File / Directory	Killing	Covering	Not Covering	Total tests
All tests	77	17	0	94
operacoes.test.js	77	17	0	94

A evolução exponencial da pontuação comprova a drástica melhoria na confiabilidade da suíte, que agora é capaz de bloquear falhas semânticas e desvios de regra de negócio com precisão.

5. Conclusão

No contexto da engenharia de software contemporânea, garantir a qualidade técnica exige mais do que bater metas superficiais de cobertura de código. Este trabalho prático demonstrou de forma contundente que testes de software precisam passar por auditorias de eficácia.

O teste de mutação provou ser uma métrica de avaliação indispensável. Ele atua como um "teste para os testes", revelando pontos cegos, falsos positivos e asserções frágeis. A incorporação dessa análise na pipeline de validação garante que a equipe de desenvolvimento não apenas crie testes, mas crie testes que possuam capacidade real de defender a integridade e a distribuição de eficácia do código base frente a regressões estruturais.